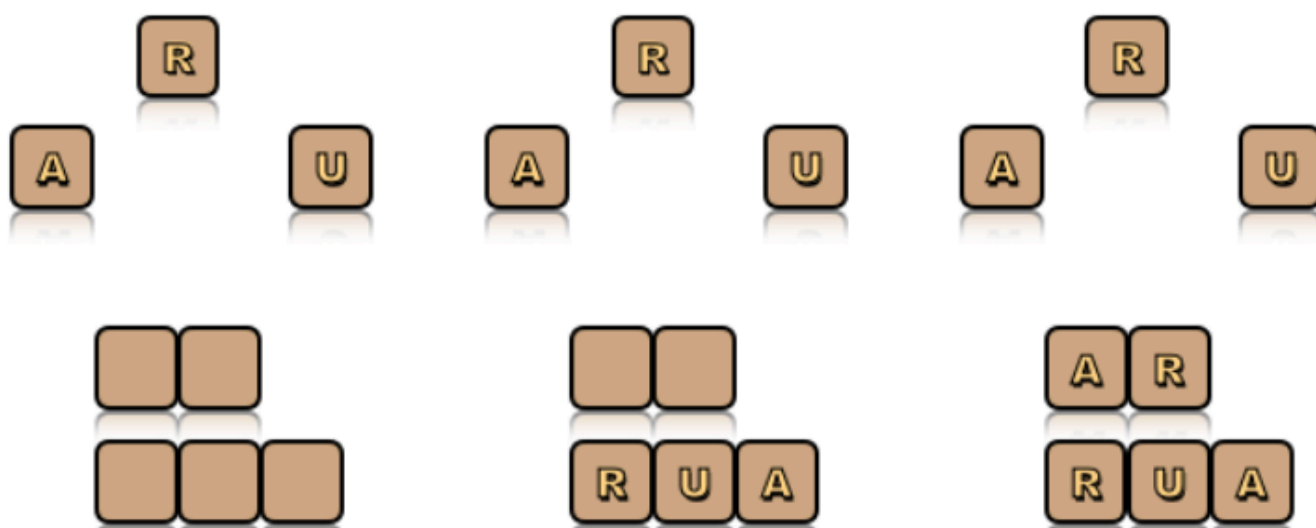


Relatório do trabalho prático

Programação II



Trabalho realizado por:

Pedro Jorge N° 54878

Diogo Tavares N° 58049

João Fltas N° 58595

Índice

Introdução.....	3
Descrição das variáveis:.....	4
allWords.....	4
remainingWords e guessedWords.....	4
allLetters e wordSizes.....	4
coins.....	5
currentLevel.....	5
filePath e saveFilePath.....	5
dictionaryPath.....	5
validWords e extraValidWords.....	6
Descrição das funções e métodos:.....	7
Método main.....	7
Função playGame.....	7
Metodo updateRemainingWords.....	7
Função clues.....	7
Função saveGameState.....	8
Método createNewGame.....	8
Método generateRandomLetters.....	9
Função findValidWords.....	9
Função isValidWord.....	9
Função loadDictionary.....	9
Função loadGameState.....	9

Introdução

Este relatório descreve o desenvolvimento de um projeto prático de Programação II, no qual foi implementado um jogo de palavras chamado GURU2P2. O objetivo principal do projeto é explorar os conceitos de programação orientada a objetos e aplicá-los utilizando a linguagem Java.

O jogo baseia-se na ideia de que o jogador deve adivinhar palavras com base em um conjunto de letras fornecido pelo programa. Cada nível apresenta um conjunto de letras e uma lista parcial das palavras possíveis de serem formadas, não previamente mostrada ao jogador mas carregada num ficheiro. O jogador avança para o próximo nível quando consegue adivinhar todas as palavras válidas para aquele nível.

O projeto foi dividido em três partes:

1. **Parte I** - Implementação do jogo base: o jogador pode jogar, salvar e carregar o progresso, além de acumular moedas ao adivinhar palavras.
2. **Parte II** - Integração com um dicionário: o sistema valida as palavras formadas com base no arquivo fornecido "portuguese-large.txt" e permite a geração automática de novos níveis.
3. **Parte III** (não implementada)- Disponibilização de uma interface gráfica para tornar o jogo mais interativo e acessível aos usuários.

Este relatório detalha os passos de implementação, as escolhas tecnológicas e as soluções adotadas para cada etapa do projeto. Além disso, são apresentados os desafios enfrentados e como foram superados, bem como as funcionalidades principais desenvolvidas no sistema.

Descrição das variáveis:

- **allWords:** Lista de todas as palavras do jogo. Esta lista é carregada através do arquivo de níveis (especificado em filePath). Cada nível do jogo possui um conjunto de palavras válidas que estão armazenadas no allWords para uso durante o progresso do jogador. A lista serve como base para determinar as palavras que podem ser adivinhadas em cada nível.
- **remainingWords e guessedWords:** Listas de Palavras pendentes e já adivinhadas. A lista remainingWords contém as palavras válidas para o nível atual que o jogador ainda não acertou e é atualizada sempre que o jogador adivinha corretamente uma palavra, removendo-a da lista de pendentes. Simultaneamente, esta palavra é adicionada à lista guessedWords, que rastreia o progresso do jogador no nível atual. Essas listas garantem que o sistema saiba quais palavras ainda precisam ser adivinhadas e quais já foram completadas.
- **allLetters e wordSizes:** Lista com letras disponíveis e tamanhos das palavras por nível. A lista allLetters é inicializada a partir do arquivo de níveis e contém as letras que o jogador pode usar para formar palavras em cada nível. Já a lista wordSizes especifica o tamanho de cada palavra que deve ser adivinhada, ajudando o jogador a identificar a estrutura das palavras esperadas.

- **coins:** Moedas acumuladas pelo jogador. As moedas são ganhas ao adivinhar corretamente uma palavra e podem ser usadas para solicitar dicas, como revelar uma letra de uma palavra pendente. Cada palavra correta concede 10 moedas, enquanto cada dica custa 100 moedas.
- **currentLevel:** Representa o nível atual em que o jogador se encontra no jogo. O valor é atualizado conforme o jogador avança, indicando o progresso geral no jogo. Este valor é usado por outras funções e métodos para saber o estado atual do jogo.
- **filePath e saveFilePath:** Caminhos para o arquivo de níveis e de salvamento. filePath especifica o caminho do arquivo que contém os níveis do jogo onde estão armazenadas as letras e palavras válidas de cada nível, que são carregadas no início do jogo ou durante a progressão de níveis. saveFilePath Determina o caminho para o arquivo de salvamento do estado do jogo. Este arquivo armazena informações sobre moedas acumuladas, o nível atual, e o progresso geral, permitindo que o jogador retome o jogo a partir do ponto onde parou.
- **dictionaryPath:** Indica o caminho para o arquivo que contém o dicionário de palavras válidas. Esse arquivo é usado para carregar uma lista de palavras permitidas que podem ser usadas no jogo, garantindo que apenas palavras válidas sejam aceitas como respostas ou processadas para gerar novos níveis.

- **validWords** e **extraValidWords**: Conjuntos de palavras válidas para validação. **validWords** é um conjunto de palavras válidas que são carregadas a partir do arquivo de dicionário. Essas palavras são usadas para verificar se as respostas fornecidas pelo jogador são válidas e também para gerar palavras nos níveis. **extraValidWords** armazena palavras válidas que o jogador digitou, mas que não fazem parte do nível atual. Quando o jogador acerta uma dessas palavras, ele ganha moedas como recompensa, embora a palavra não contribua para completar o nível.

Descrição das funções e métodos:

Método main

- **Descrição:** O método main é o ponto de entrada do programa e realiza as seguintes ações:
 1. Pergunta ao jogador se ele deseja carregar um jogo salvo anteriormente, iniciar um novo jogo ou criar novos níveis.
 2. Recebe a escolha do jogador e executa a ação correspondente:
 - **Opção 1:** Reinicia todas as variáveis do estado do jogo e retorna ao nível 1.
 - **Opção 2:** Carrega um jogo salvo usando o método `loadGame()`.
 - **Opção 3:** Cria um novo conjunto de níveis com o método `createNewGame()`.
 3. Inicia o método `playGame()` para começar o jogo de acordo com a escolha feita.

Função `playGame(String filePath, String saveFilePath)`

- **Argumentos:**
 - `filePath`: String com o caminho para o arquivo contendo os níveis do jogo, no nosso caso o ficheiro *ficheiro_níveis.txt*
 - `saveFilePath`: String com o caminho para salvar o estado do jogo.

- **Descrição:**
 - Accede ao caminho presente na string filePath com um Scanner
 - Lê os dados do arquivo encontrado no caminho e converte-os em níveis.
 - Gerencia o fluxo do jogo:
 - Apresenta letras e palavras para adivinhar.
 - Aceita entradas do jogador para adivinhar palavras, pedir dicas ou salvar o progresso.
 - Atualiza todo o sistema de moedas e o progresso do nível.
 - Permite salvar o estado do jogo usando saveGameState.
-

Metodo updateRemainingWords()

- **Descrição:**
 - Atualiza a lista remainingWords removendo palavras que já foram adivinhadas.
 - Garante que remainingWords contenha apenas palavras pendentes do nível atual.
-

Função clues(int wordIndex, int letterIndex, List<String> wordsToBeGuessed)

- **Argumentos:**
 - wordIndex: Índice da palavra na lista de palavras restantes.
 - letterIndex: Índice da letra dentro da palavra.
 - wordsToBeGuessed: Lista de strings contendo as palavras restantes no nível atual.
- **Descrição:**
 - Gerencia todo o sistema de pistas
 - Valida os índices fornecidos.
 - Retorna a letra da palavra correspondente aos índices fornecidos pelo jogador.
 - Caso os índices sejam inválidos, retorna mensagens de erro.

Função saveGameState(String saveFilePath, String originalGameFilePath)

- **Argumentos:**
 - saveFilePath: String com o caminho para o arquivo onde o estado do jogo será salvo.
 - originalGameFilePath: String com o caminho do arquivo original com os níveis do jogo.
 - **Descrição:**
 - Salva moedas e o nível atual no arquivo especificado.
 - Armazena palavras restantes nos níveis que ainda não foram completados.
-

Método createNewGame()

- **Descrição:**
 - Solicita ao jogador o número de níveis a serem gerados e o nome do arquivo onde os níveis serão salvos.
 - Gera letras aleatórias usando generateRandomLetters.
 - Cria níveis contendo palavras válidas a partir do dicionário fornecido com base nas letras geradas.
 - Escreve os níveis no formato apropriado em um arquivo.

Método generateRandomLetters()

- **Descrição:**
 - Gera um conjunto aleatório de letras, garantindo pelo menos uma vogal.
 - Mistura vogais e consoantes aleatoriamente com o método random
-

Função findValidWords(List<String> letters)

- **Argumentos:**
 - letters: Lista de strings contendo letras disponíveis para formar palavras.
 - **Descrição:**
 - Busca palavras válidas no dicionário que podem ser formadas com as letras fornecidas.
 - Retorna uma lista de palavras possíveis.
-

Função isWordValid(String word, List<String> letters)

- **Argumentos:**

- word: Palavra a ser validada.
 - letters: Lista de letras disponíveis.
 - **Descrição:**
 - Verifica se a palavra pode ser formada com as letras fornecidas, levando em consideração a quantidade de cada letra.
 - Retorna true se a palavra for válida; caso contrário, retorna false.
-

Função loadDictionary(String dictionaryPath)

- **Argumentos:**
 - dictionaryPath: Caminho para o arquivo contendo o dicionário.
 - **Descrição:**
 - Lê o arquivo linha por linha.
 - Adiciona as palavras ao conjunto validWords em letras maiúsculas para facilitar a comparação.
-

Função loadGameState(String saveFilePath, String originalGameFilePath)

- **Argumentos:**
 - saveFilePath: Caminho para o arquivo onde o estado do jogo foi salvo.
 - originalGameFilePath: Caminho do arquivo original com os níveis do jogo.
- **Descrição:**
 - Lê moedas e o nível atual do arquivo salvo.
 - Caso ocorra um erro, inicia um novo jogo com configurações padrão.
- **Função:** Lê o ficheiro de níveis (linha por linha). Cada nível começa com uma linha contendo as letras disponíveis, seguidas pelas palavras válidas que o jogador deve adivinhar.